

Capstone

July 21, 2026

1 Capstone: Define and Solve an ML Problem

```
[1]: import pandas as pd
import numpy as np
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
import tensorflow.keras as keras
from sklearn.preprocessing import StandardScaler
import time
```

Note: As you work through the notebook, you can import additional packages as needed.

1.1 Overview

In this capstone assignment, you will follow the machine learning life cycle and implement one of the supervised learning models you have learned so far in this course, along with a neural network, to solve a predictive problem.

This capstone spans two lab sessions.

- **Unit 5 Lab:** You will define a machine learning problem, explore and prepare your data, and train, test, evaluate and improve a traditional machine learning model (Parts 1–5).
- **Unit 6 Lab:** After completing Unit 6 on neural networks, you will apply a neural network to the same problem and compare the two approaches (Parts 6–7).

There is a checkpoint at the end of Part 5 that marks where to stop during the Unit 5 lab.

You will complete the following:

1. Choose your Data Set and Build Your DataFrame
2. Define Your ML Problem

3. Understand Your Data
4. Prepare Your Data
5. Train, Test, Evaluate and Improve a Traditional Machine Learning Model (*Unit 5 lab*)
6. Train, Test, Evaluate and Improve Neural Network (*Unit 6 lab*)
7. Compare Your Models and Reflect (*Unit 6 lab*)

This is an individual assignment. You are welcome to discuss ideas with your peers, but the code and written responses you submit must be your own.

Note: This capstone is intentionally less scaffolded than your weekly labs; that is by design. You are expected to make your own implementation choices, add code cells as needed, and document your reasoning throughout.

1.2 Part 1: Choose Your Data Set and Build Your DataFrame

You will choose one of two data sets to work with for this capstone. In both cases, you will be solving a supervised learning binary classification problem by predicting one of two possible class labels. Both data sets have been used earlier in the course, so you are already familiar with their structure.

Option A: Census Income Data (`censusData.csv`) This data set contains demographic and employment information from the 1994 U.S. Census. You will use it to predict whether an individual's annual income exceeds \$50,000. Your label column is `income_binary`, which contains two values: `<=50K` and `>50K`. You will need to convert this column into a binary numeric label (for example, 0 and 1) during data preparation.

Option B: Airbnb NYC Listings Data (`airbnbListingsData.csv`) This data set contains information about Airbnb listings in New York City. You will use it to predict whether a listing is high-priced. The data set includes a new `price_category` column that classifies each listing as either "high price" or "low price" based on whether the listing's price falls above or below the 75th percentile of all listing prices. Listings at or above the 75th percentile are labeled as `high`, while the remaining listings are labeled as `low`. You will need to convert this column into a binary numeric label (for example, 0 and 1) during data preparation.

Note: These versions of the data sets differ slightly from the versions you have worked with in this program. For example, they may not include some of the preprocessing necessary for specific models.

Load a Data Set and Save it as a Pandas DataFrame The code cell below contains filenames (`path + filename`) for the two data sets available to you.

Task: In the code cell below, load your chosen data set using `pd.read_csv()` and save it to a DataFrame named `df`. Then call `df.head()` to inspect the first few row of the data set.

```
[51]: # File paths for both data sets
census_filename = os.path.join(os.getcwd(), "data_capstone", "censusData.csv")
airbnb_filename = os.path.join(os.getcwd(), "data_capstone", "airbnbListingsData.csv")
```

```
# Load your chosen dataset and save it to df
df = pd.read_csv(airbnb_filename)

df.head()
```

```
[51]:
```

	name \		description \	neighborhood_overview	host_name \	host_location \	host_about	host_response_rate \	host_acceptance_rate	host_is_superhost	host_listings_count	...	\
0	Skylit Midtown Castle		Beautiful, spacious skylit studio in the heart...	Centrally located in the heart of Manhattan ju...	Jennifer	New York, New York, United States	A New Yorker since 2000! My passion is creatin...	0.80	0.17	True	8.0	...	
1	Whole flr w/private bdrm, bath & kitchen(pls r...		Enjoy 500 s.f. top floor in 1899 brownstone, w...	Just the right mix of urban center and local n...	LisaRoxanne	New York, New York, United States	Laid-back Native New Yorker (formerly bi-coast...	0.09	0.69	True	1.0	...	
2	Spacious Brooklyn Duplex, Patio + Garden		We welcome you to stay in our lovely 2 br dupl...		NaN	Brooklyn, New York, United States	Rebecca is an artist/designer, and Henoch is i...	1.00	0.25	True	1.0	...	
3	Large Furnished Room Near B'way		Please don't expect the luxury here just a bas...	Theater district, many restaurants around here.	Shunichi	New York, New York, United States	I used to work for a financial industry but no...	1.00	1.00	True	1.0	...	
4	Cozy Clean Guest Room - Family Apt		Our best guests are seeking a safe, clean, spa...	Our neighborhood is full of restaurants and ca...	MaryEllen	New York, New York, United States	Welcome to family life with my oldest two away...	NaN	NaN	True	1.0	...	

	review_scores_location	review_scores_value	instant_bookable	\
0	4.86	4.41	False	
1	4.71	4.64	False	
2	4.50	5.00	False	
3	4.87	4.36	False	
4	4.94	4.92	False	

	calculated_host_listings_count	calculated_host_listings_count_entire_homes	\
0	3	3	
1	1	1	
2	1	1	
3	1	0	
4	1	0	

	calculated_host_listings_count_private_rooms	\
0	0	
1	0	
2	0	
3	1	
4	1	

	calculated_host_listings_count_shared_rooms	reviews_per_month	\
0	0	0.33	
1	0	4.86	
2	0	0.02	
3	0	3.68	
4	0	0.87	

	n_host_verifications	price_category
0	9	low
1	6	low
2	3	high
3	4	low
4	7	low

[5 rows x 51 columns]

```
[52]: df.columns
```

```
[52]: Index(['name', 'description', 'neighborhood_overview', 'host_name',
        'host_location', 'host_about', 'host_response_rate',
        'host_acceptance_rate', 'host_is_superhost', 'host_listings_count',
        'host_total_listings_count', 'host_has_profile_pic',
        'host_identity_verified', 'neighbourhood_group_cleansed', 'room_type',
        'accommodates', 'bathrooms', 'bedrooms', 'beds', 'amenities', 'price',
        'minimum_nights', 'maximum_nights', 'minimum_minimum_nights',
        'maximum_minimum_nights', 'minimum_maximum_nights',
```

```
'maximum_maximum_nights', 'minimum_nights_avg_ntm',
'maximum_nights_avg_ntm', 'has_availability', 'availability_30',
'availability_60', 'availability_90', 'availability_365',
'number_of_reviews', 'number_of_reviews_ltm', 'number_of_reviews_l30d',
'review_scores_rating', 'review_scores_cleanliness',
'review_scores_checkin', 'review_scores_communication',
'review_scores_location', 'review_scores_value', 'instant_bookable',
'calculated_host_listings_count',
'calculated_host_listings_count_entire_homes',
'calculated_host_listings_count_private_rooms',
'calculated_host_listings_count_shared_rooms', 'reviews_per_month',
'n_host_verifications', 'price_category'],
dtype='object')
```

1.3 Part 2: Define Your ML Problem

The first step of the machine learning life cycle involves defining your business problem. In the markdown cell below, you will clearly define what you are trying to predict and why it matters.

Task: In the markdown cell below, answer all of the following questions:

1. Which data set did you choose?
 2. What is your label? What are you predicting?
 3. What features do you plan to use? (This list may change after you explore your data.)
 4. Why does this problem matter? Using the business brief you read in the lab overview page, explain how the organization described there could use a model that predicts this label to create value or inform decisions for their client.
-
1. I chose Airbnb NYC Listings Data set
 2. The label is price_category. This model will predict whether a listing is high-priced.
 3. The features I plan to use are host_listings_count, host_total_listings_count, host_is_superhost, room_type, accommodates, bathrooms, bedrooms, beds, amenities, etc
 4. This problem matters because the prediction results from the model can help maximize yield and revenue and booking instead of focusing on filling empty spots. The organization described in the business brief could use this model to advise hosts on when and how to adjust their prices based on how competitive their listing is relative to the market.

1.4 Part 3: Understand Your Data

Now that you have defined your problem, perform exploratory data analysis (EDA) with that problem in mind. Consider the following as you inspect your data:

1. What data preparation techniques would you like to use? These data preparation techniques may include:
 - handling missing values
 - finding and replacing outliers

- performing feature engineering techniques such as one-hot encoding on categorical features
 - selecting appropriate features and removing irrelevant features
 - addressing class imbalance
2. What machine learning model would you like to use that is suitable for your predictive problem and data?
 - You may use one of the following models that you have worked with so far: - K-Nearest Neighbors (KNN) - Decision Tree - Logistic Regression
 - Are there other data preparation techniques that you will need to apply to build a balanced modeling data set for your problem and model? For example, will you need to scale your data?
 3. How will you evaluate and improve the model's performance?
 - Are there specific evaluation metrics or methods that are appropriate for your problem, dataset, or selected model?

Task: In the code cells below, use the techniques you have learned in this course to inspect and analyze your data.

Note: You can add code cells if needed by going to the Insert menu and clicking on Insert Cell Below in the drop-down menu.

```
[3]: nan_count = np.sum(df.isnull(), axis = 0)
     nan_count
```

```
[3]: name                    5
     description              570
     neighborhood_overview    9816
     host_name                0
     host_location            60
     host_about              10945
     host_response_rate       11843
     host_acceptance_rate     11113
     host_is_superhost        0
     host_listings_count      0
     host_total_listings_count 0
     host_has_profile_pic     0
     host_identity_verified   0
     neighbourhood_group_cleansed 0
     room_type                0
     accommodates             0
     bathrooms                0
     bedrooms                 2918
     beds                    1354
     amenities                0
     price                    0
     minimum_nights           0
     maximum_nights           0
```

minimum_minimum_nights	0
maximum_minimum_nights	0
minimum_maximum_nights	0
maximum_maximum_nights	0
minimum_nights_avg_ntm	0
maximum_nights_avg_ntm	0
has_availability	0
availability_30	0
availability_60	0
availability_90	0
availability_365	0
number_of_reviews	0
number_of_reviews_ltm	0
number_of_reviews_l30d	0
review_scores_rating	0
review_scores_cleanliness	0
review_scores_checkin	0
review_scores_communication	0
review_scores_location	0
review_scores_value	0
instant_bookable	0
calculated_host_listings_count	0
calculated_host_listings_count_entire_homes	0
calculated_host_listings_count_private_rooms	0
calculated_host_listings_count_shared_rooms	0
reviews_per_month	0
n_host_verifications	0
price_category	0
dtype: int64	

```
[4]: nan_detected = nan_count > 0
      nan_detected
```

[4]: name	True
description	True
neighborhood_overview	True
host_name	False
host_location	True
host_about	True
host_response_rate	True
host_acceptance_rate	True
host_is_superhost	False
host_listings_count	False
host_total_listings_count	False
host_has_profile_pic	False
host_identity_verified	False
neighbourhood_group_cleansed	False

room_type	False
accommodates	False
bathrooms	False
bedrooms	True
beds	True
amenities	False
price	False
minimum_nights	False
maximum_nights	False
minimum_minimum_nights	False
maximum_minimum_nights	False
minimum_maximum_nights	False
maximum_maximum_nights	False
minimum_nights_avg_ntm	False
maximum_nights_avg_ntm	False
has_availability	False
availability_30	False
availability_60	False
availability_90	False
availability_365	False
number_of_reviews	False
number_of_reviews_ltm	False
number_of_reviews_l30d	False
review_scores_rating	False
review_scores_cleanliness	False
review_scores_checkin	False
review_scores_communication	False
review_scores_location	False
review_scores_value	False
instant_bookable	False
calculated_host_listings_count	False
calculated_host_listings_count_entire_homes	False
calculated_host_listings_count_private_rooms	False
calculated_host_listings_count_shared_rooms	False
reviews_per_month	False
n_host_verifications	False
price_category	False
dtype: bool	

```
[5]: is_int_or_float = (df.dtypes == "int64") | (df.dtypes == "float64")
is_int_or_float
```

[5]: name	False
description	False
neighborhood_overview	False
host_name	False
host_location	False

host_about	False
host_response_rate	True
host_acceptance_rate	True
host_is_superhost	False
host_listings_count	True
host_total_listings_count	True
host_has_profile_pic	False
host_identity_verified	False
neighbourhood_group_cleansed	False
room_type	False
accommodates	True
bathrooms	True
bedrooms	True
beds	True
amenities	False
price	True
minimum_nights	True
maximum_nights	True
minimum_minimum_nights	True
maximum_minimum_nights	True
minimum_maximum_nights	True
maximum_maximum_nights	True
minimum_nights_avg_ntm	True
maximum_nights_avg_ntm	True
has_availability	False
availability_30	True
availability_60	True
availability_90	True
availability_365	True
number_of_reviews	True
number_of_reviews_ltm	True
number_of_reviews_l30d	True
review_scores_rating	True
review_scores_cleanliness	True
review_scores_checkin	True
review_scores_communication	True
review_scores_location	True
review_scores_value	True
instant_bookable	False
calculated_host_listings_count	True
calculated_host_listings_count_entire_homes	True
calculated_host_listings_count_private_rooms	True
calculated_host_listings_count_shared_rooms	True
reviews_per_month	True
n_host_verifications	True
price_category	False
dtype: bool	

```
[6]: to_impute = nan_detected & is_int_or_float
     to_impute
```

```
[6]: name                    False
     description              False
     neighborhood_overview    False
     host_name                 False
     host_location            False
     host_about               False
     host_response_rate       True
     host_acceptance_rate     True
     host_is_superhost        False
     host_listings_count      False
     host_total_listings_count False
     host_has_profile_pic     False
     host_identity_verified   False
     neighbourhood_group_cleansed False
     room_type                False
     accommodates             False
     bathrooms                False
     bedrooms                 True
     beds                    True
     amenities                False
     price                    False
     minimum_nights           False
     maximum_nights           False
     minimum_minimum_nights   False
     maximum_minimum_nights   False
     minimum_maximum_nights   False
     maximum_maximum_nights   False
     minimum_nights_avg_ntm   False
     maximum_nights_avg_ntm   False
     has_availability          False
     availability_30           False
     availability_60           False
     availability_90           False
     availability_365          False
     number_of_reviews        False
     number_of_reviews_ltm     False
     number_of_reviews_l30d    False
     review_scores_rating      False
     review_scores_cleanliness False
     review_scores_checkin     False
     review_scores_communication False
     review_scores_location    False
     review_scores_value       False
     instant_bookable          False
```

calculated_host_listings_count	False
calculated_host_listings_count_entire_homes	False
calculated_host_listings_count_private_rooms	False
calculated_host_listings_count_shared_rooms	False
reviews_per_month	False
n_host_verifications	False
price_category	False
dtype: bool	

```
[7]: df.columns[to_impute]
```

```
[7]: Index(['host_response_rate', 'host_acceptance_rate', 'bedrooms', 'beds'],
dtype='object')
```

```
[8]: cate_values = list(df.select_dtypes(include = ['object']).columns)
df[cate_values].nunique()
cate_values
```

```
[8]: ['name',
      'description',
      'neighborhood_overview',
      'host_name',
      'host_location',
      'host_about',
      'neighbourhood_group_cleansed',
      'room_type',
      'amenities',
      'price_category']
```

```
[9]: for col in cate_values:
      print(f"{col} has {df[col].nunique()} unique values")
```

```
name has 27386 unique values
description has 25952 unique values
neighborhood_overview has 15800 unique values
host_name has 7566 unique values
host_location has 1364 unique values
host_about has 11962 unique values
neighbourhood_group_cleansed has 5 unique values
room_type has 4 unique values
amenities has 25020 unique values
price_category has 2 unique values
```

```
[10]: df['neighbourhood_group_cleansed'].unique()
df['room_type'].unique()
```

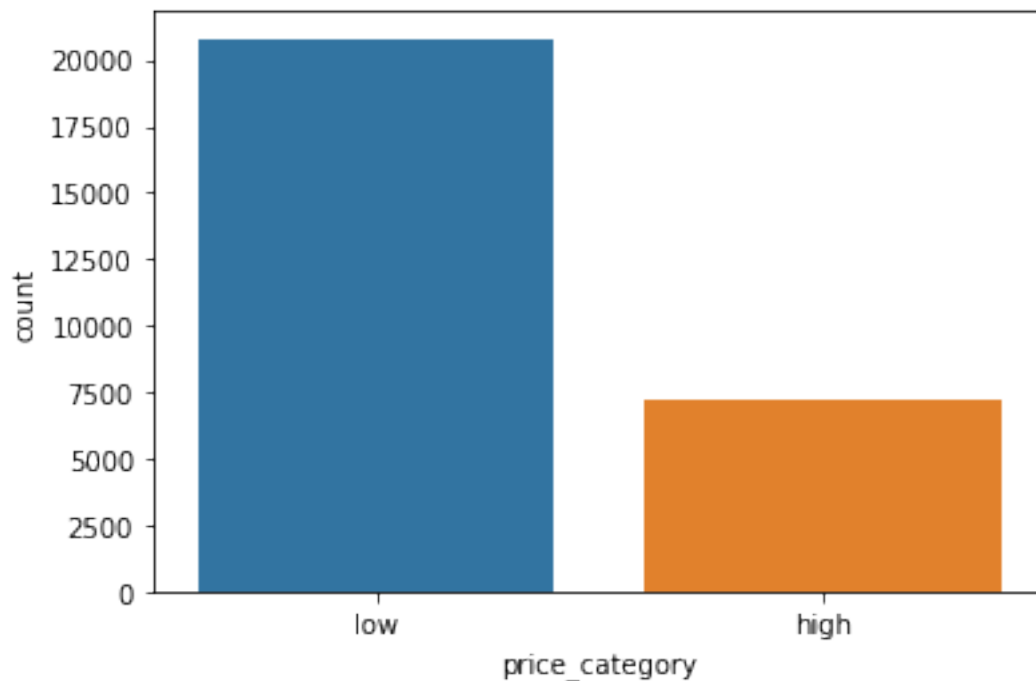
```
[10]: array(['Entire home/apt', 'Private room', 'Hotel room', 'Shared room'],  
        dtype=object)
```

1.4.1 Class Imbalance

Examine the distribution of your label column to determine whether class imbalance is present. Use at least one visualization to show the class distribution. In the **EDA Summary** below, you will discuss how you plan to address any observed imbalance during data preparation.

```
[11]: df['price_category'].value_counts(normalize=True)  
sns.countplot(x=df['price_category'])
```

```
[11]: <AxesSubplot:xlabel='price_category', ylabel='count'>
```



1.4.2 Inspect and Analyze Your Data

Explore your features. Use summary statistics and visualizations to understand how your features are distributed and how they relate to the label. Identify issues such as missing values, outliers, or a feature that is irrelevant or redundant.

Think of the different techniques you have used to inspect and analyze your data in this course. These include using Pandas to apply data filters, using the Pandas `describe()` method to get insight into key statistics for each column, using the Pandas `dtypes` property to inspect the data

type of each column, and using Matplotlib and Seaborn to detect outliers and visualize relationships between features and labels.

Use at least one plot that visualizes a relationship between features and the label.

```
[12]: df.describe()
```

```
[12]:
```

	host_response_rate	host_acceptance_rate	host_listings_count	\
count	16179.000000	16909.000000	28022.000000	
mean	0.906901	0.791953	14.554778	
std	0.227282	0.276732	120.721287	
min	0.000000	0.000000	0.000000	
25%	0.940000	0.680000	1.000000	
50%	1.000000	0.910000	1.000000	
75%	1.000000	1.000000	3.000000	
max	1.000000	1.000000	3387.000000	

	host_total_listings_count	accommodates	bathrooms	bedrooms	\
count	28022.000000	28022.000000	28022.000000	25104.000000	
mean	14.554778	2.874491	1.142174	1.329708	
std	120.721287	1.860251	0.421132	0.700726	
min	0.000000	1.000000	0.000000	1.000000	
25%	1.000000	2.000000	1.000000	1.000000	
50%	1.000000	2.000000	1.000000	1.000000	
75%	3.000000	4.000000	1.000000	1.000000	
max	3387.000000	16.000000	8.000000	12.000000	

	beds	price	minimum_nights	...	review_scores_checkin	\
count	26668.000000	28022.000000	28022.000000	...	28022.000000	
mean	1.629556	154.228749	18.689387	...	4.814300	
std	1.097104	140.816605	25.569151	...	0.438603	
min	1.000000	29.000000	1.000000	...	0.000000	
25%	1.000000	70.000000	2.000000	...	4.810000	
50%	1.000000	115.000000	30.000000	...	4.960000	
75%	2.000000	180.000000	30.000000	...	5.000000	
max	21.000000	1000.000000	1250.000000	...	5.000000	

	review_scores_communication	review_scores_location	\
count	28022.000000	28022.000000	
mean	4.808041	4.750393	
std	0.464585	0.415717	
min	0.000000	0.000000	
25%	4.810000	4.670000	
50%	4.970000	4.880000	
75%	5.000000	5.000000	
max	5.000000	5.000000	

	review_scores_value	calculated_host_listings_count	\
--	---------------------	--------------------------------	---

count	28022.000000	28022.000000
mean	4.647670	9.581900
std	0.518023	32.227523
min	0.000000	1.000000
25%	4.550000	1.000000
50%	4.780000	1.000000
75%	5.000000	3.000000
max	5.000000	421.000000

	calculated_host_listings_count_entire_homes \
count	28022.000000
mean	5.562986
std	26.121426
min	0.000000
25%	0.000000
50%	1.000000
75%	1.000000
max	308.000000

	calculated_host_listings_count_private_rooms \
count	28022.000000
mean	3.902077
std	17.972386
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	359.000000

	calculated_host_listings_count_shared_rooms	reviews_per_month \
count	28022.000000	28022.000000
mean	0.048283	1.758325
std	0.442459	4.446143
min	0.000000	0.010000
25%	0.000000	0.130000
50%	0.000000	0.510000
75%	0.000000	1.830000
max	8.000000	141.000000

	n_host_verifications
count	28022.000000
mean	5.169510
std	2.028497
min	1.000000
25%	4.000000
50%	5.000000
75%	7.000000

max 13.000000

[8 rows x 36 columns]

```
[13]: # relation between features and the label
df['price_category_num'] = df['price_category'].map({'low': 0, 'high': 1})

corr_matrix = round(df.corr(), 5)
corrs = corr_matrix['price_category_num']
corrs_sorted = corrs.sort_values(ascending=False)
corrs_sorted
```

```
[13]: price_category_num    1.00000
price                    0.70734
accommodates             0.43171
bedrooms                 0.40452
beds                    0.34384
bathrooms               0.21570
availability_60          0.10725
availability_90          0.10614
availability_30          0.09545
host_total_listings_count 0.08951
host_listings_count      0.08951
review_scores_location    0.08918
availability_365          0.08154
review_scores_cleanliness 0.07455
maximum_maximum_nights    0.06348
number_of_reviews_ltm     0.06003
has_availability          0.05917
number_of_reviews_130d    0.05349
maximum_nights_avg_ntm    0.05257
calculated_host_listings_count_entire_homes 0.05081
host_acceptance_rate      0.04471
review_scores_rating      0.04239
reviews_per_month         0.03758
instant_bookable          0.03715
minimum_maximum_nights    0.03612
calculated_host_listings_count 0.03316
host_response_rate        0.01951
maximum_minimum_nights    0.01233
minimum_nights_avg_ntm    0.00916
review_scores_communication 0.00338
review_scores_checkin     -0.00326
maximum_nights            -0.00359
review_scores_value       -0.00429
n_host_verifications      -0.01189
calculated_host_listings_count_private_rooms -0.01556
```

```

number_of_reviews                    -0.03558
calculated_host_listings_count_shared_rooms -0.04357
minimum_minimum_nights              -0.05910
minimum_nights                      -0.06695
host_is_superhost                   NaN
host_has_profile_pic                NaN
host_identity_verified              NaN
Name: price_category_num, dtype: float64

```

```

[14]: # investigating the NaN values
df[['host_is_superhost', 'host_has_profile_pic', 'host_identity_verified']].
    ↳dtypes

```

```

[14]: host_is_superhost      bool
      host_has_profile_pic  bool
      host_identity_verified bool
      dtype: object

```

```

[15]: bool_cols = ['host_is_superhost', 'host_has_profile_pic',
    ↳'host_identity_verified']
df[bool_cols] = df[bool_cols].astype(int)
corr_matrix = round(df.corr(), 5)
corr_matrix['price_category_num'][bool_cols]

```

```

[15]: host_is_superhost      NaN
      host_has_profile_pic  NaN
      host_identity_verified NaN
      Name: price_category_num, dtype: float64

```

```

[16]: df[bool_cols].nunique()

```

```

[16]: host_is_superhost      1
      host_has_profile_pic   1
      host_identity_verified  1
      dtype: int64

```

```

[17]: top_two_corr = list(corr_sorted.drop(['price_category_num'])[:2].index)
      top_two_corr

```

```

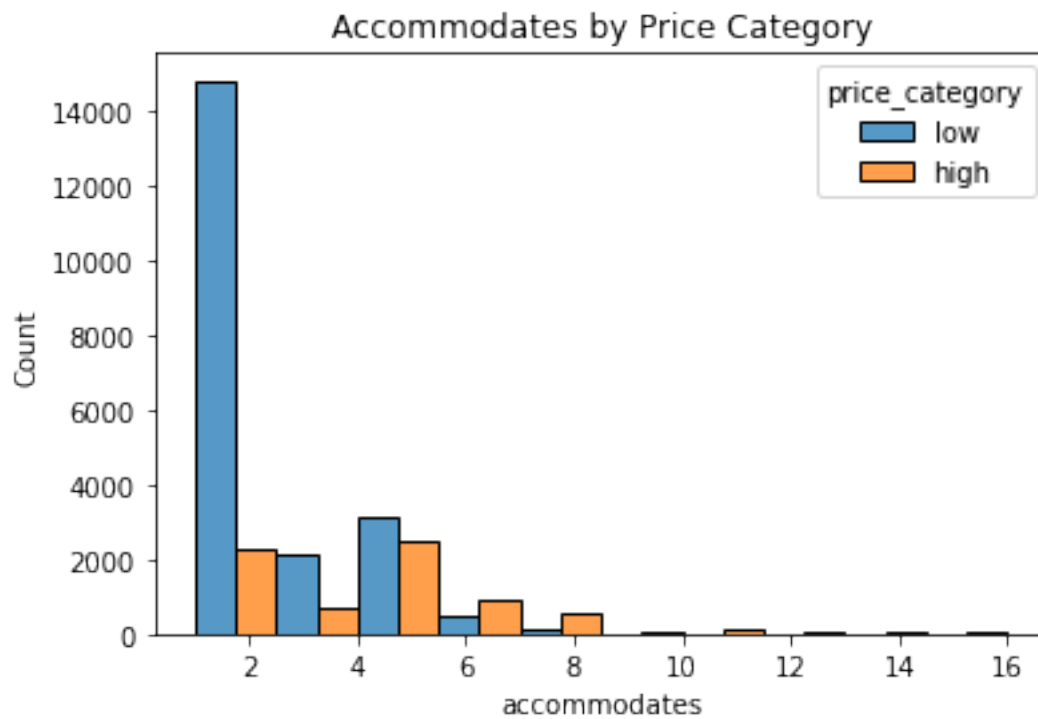
[17]: ['price', 'accommodates']

```

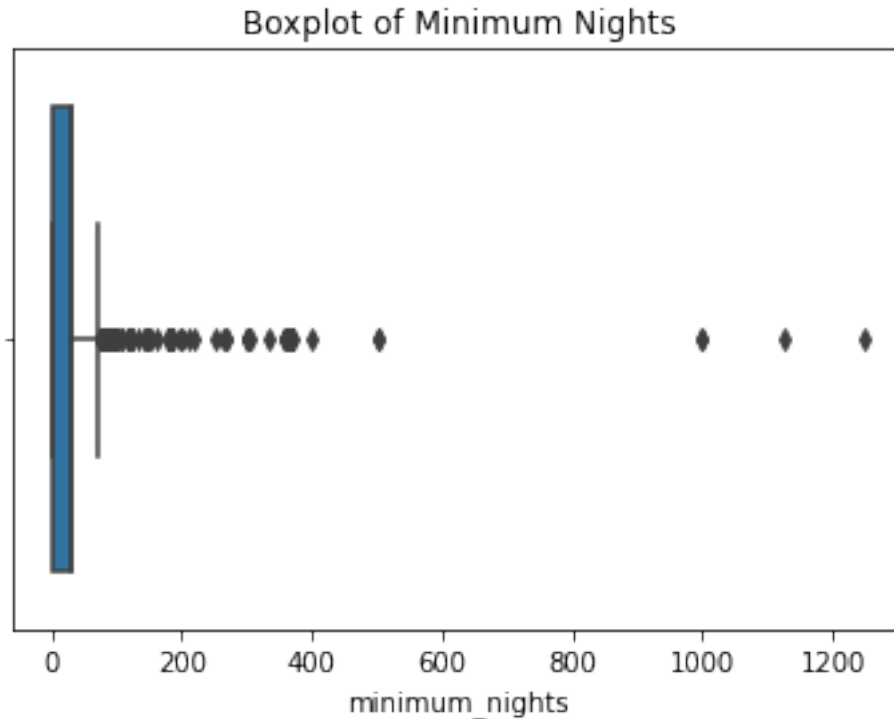
```

[18]: # visualizaton of the relationship between a feature and the label
sns.histplot(data=df, x='accommodates', hue='price_category', multiple='dodge',
    ↳bins=10)
plt.title('Accommodates by Price Category')
plt.show()

```

```
[19]: # outlier detection
plt.figure(figsize=(6,4))
sns.boxplot(x=df["minimum_nights"])
plt.title("Boxplot of Minimum Nights")
plt.show()
```



1.4.3 EDA Summary

Task: In the markdown cell below, summarize the key findings from your data exploration. Describe any patterns, anomalies, or data quality issues you identified and explain how those findings may influence your data preparation decisions. For example, your exploration may affect how you handle missing values, address class imbalance, or determine which features to keep or remove.

I chose the Airbnb NYC Listings dataset, with `price_category` as my label. My goal is to predict whether a listing will be high-priced based on its characteristics.

Missing values: Four numeric columns had missing data: `host_response_rate`, `host_acceptance_rate`, `bedrooms`, and `beds`.

Class imbalance: I checked the distribution of `price_category` using `value_counts(normalize=True)` and a countplot. The countplot indicates a class imbalance, with 'low' being the majority class (~75%). To prevent the Decision Tree from becoming biased toward predicting the majority class, I will address this imbalance during the modeling phase by setting `class_weight='balanced'` in the model parameters, which helps the model focus on correctly identifying the less common group.

Outliers: The boxplots and descriptive statistics revealed extreme outliers in several numerical features. For example, `minimum_nights` had unrealistic values, 1000+ nights with most listings in 30 and `maximum_nights` had values in the billions.

Feature relationships: Using a correlation matrix against the label (`price_category_num`), I

found 'accommodates' (0.43171), 'bedrooms' (0.38188), 'room_type: Entire home/apt' (0.34142), and 'beds' (0.33664) were most strongly correlated with price category. I used a histogram to visualize the relationship between the feature(accommodates) and the label(price_category), showing that higher-priced listings usually fit more guests.

Redundant and irrelevant features: 'host_listings_count'(0.08951) and 'host_total_listings_count'(0.08951) were perfectly correlated with each other. Features like 'host_is_superhost', 'host_has_profile_pic', and 'host_identity_verified' all had only single unique value across the entire dataset (zero variance), which is why they returned NaN in the correlation matrix (little to no predictive power). The price column showed a correlation of 0.70734 with the label, since it was the exact variable used to create price_category. Columns like 'name', 'description', 'neighborhood_overview', 'host_name', 'host_location', 'host_about', 'amenities' had too many unique values to be usable as-is without additional processing (Natural Language Processing).

1.4.4 Ethical Considerations:

Machine learning models can cause harm when they reflect or amplify biases in the data they are trained on.

Task: In the markdown cell below, answer both of the following questions:

1. What biases or ethical concerns might be present in your dataset? Think about who collected the data, how it was collected, and which groups of people appear in it. Are there features in the dataset that could serve as proxies for protected characteristics like race, gender, or socioeconomic status?
 2. Who could be harmed by a model that makes incorrect predictions on this data, and how? Be specific about which groups are most at risk and what the real-world consequences of errors might look like.
-
1. The dataset may contain historical biases reflecting pricing discrimination based on race or gender, and socioeconomic disparities present in the New York City rental market. Yes, the neighbourhood_group_cleansed feature can act as a strong proxy for race and socioeconomic status due to historical residential segregation in New York City boroughs.
 2. Hosts and guests could be harmed if the model makes incorrect predictions, leading to unrealistic pricing recommendations. Hosts in low income neighborhoods are most at risk since incorrect low price predictions could lead to a significant reduction in income.

1.5 Part 4: Prepare Your Data

Task: In the code cell below, prepare your data for modeling. The specific steps you take will depend on what you found during your EDA and which model you plan to use.

Note: You can add code cells if needed by going to the Insert menu and clicking on Insert Cell Below in the drop-down menu.

```
[20]: to_impute_sel = ['host_response_rate', 'host_acceptance_rate', 'bedrooms',  
    ↪ 'beds']  
for col in to_impute_sel:
```

```
value = col + '_na'
df[value] = df[col].isnull()
```

```
[21]: for col in to_impute_sel:
      mean_col = df[col].mean()
      df[col] = df[col].fillna(mean_col)
```

```
[22]: for colname in to_impute_sel:
      print("{} missing values count :{}".format(colname, np.sum(df[colname].
      ↪isnull(), axis = 0)))
```

```
host_response_rate missing values count :0
host_acceptance_rate missing values count :0
bedrooms missing values count :0
beds missing values count :0
```

```
[23]: df_neighbourhood_group_cleansed = pd.
      ↪get_dummies(df['neighbourhood_group_cleansed'],
                  prefix='neighbourhood_group_cleansed: ',
      ↪prefix_sep='')
      df_neighbourhood_group_cleansed
```

```
[23]: neighbourhood_group_cleansed: Bronx \
0      0
1      0
2      0
3      0
4      0
...
28017      0
28018      0
28019      0
28020      0
28021      0

neighbourhood_group_cleansed: Brooklyn \
0      0
1      1
2      1
3      0
4      0
...
28017      0
28018      1
28019      1
28020      1
28021      0
```

```

    neighbourhood_group_cleansed: Manhattan \
0                                     1
1                                     0
2                                     0
3                                     1
4                                     1
...
28017                                0
28018                                0
28019                                0
28020                                0
28021                                0

```

```

    neighbourhood_group_cleansed: Queens \
0                                     0
1                                     0
2                                     0
3                                     0
4                                     0
...
28017                                1
28018                                0
28019                                0
28020                                0
28021                                1

```

```

    neighbourhood_group_cleansed: Staten Island
0                                     0
1                                     0
2                                     0
3                                     0
4                                     0
...
28017                                0
28018                                0
28019                                0
28020                                0
28021                                0

```

```
[28022 rows x 5 columns]
```

```
[24]: df = df.join(df_neighbourhood_group_cleansed)
df.drop(columns = 'neighbourhood_group_cleansed', inplace=True)
```

```
[25]: df_room_type = pd.get_dummies(df['room_type'], prefix='room_type: ',
    ↪ prefix_sep='')

```

```
df_room_type
```

```
[25]:      room_type: Entire home/apt  room_type: Hotel room  \
0                                1                0
1                                1                0
2                                1                0
3                                0                0
4                                0                0
...                               ...                ...
28017                           0                0
28018                           1                0
28019                           0                0
28020                           1                0
28021                           0                0
```

```
      room_type: Private room  room_type: Shared room
0                             0                0
1                             0                0
2                             0                0
3                             1                0
4                             1                0
...                           ...                ...
28017                         1                0
28018                         0                0
28019                         1                0
28020                         0                0
28021                         1                0
```

```
[28022 rows x 4 columns]
```

```
[26]: df = df.join(df_room_type)
df.drop(columns = 'room_type', inplace=True)
```

```
[27]: irr_cols = ['name', 'description', 'neighborhood_overview', 'host_name',
↳ 'host_location', 'host_about', 'amenities']
```

```
[28]: df.drop(columns=irr_cols, inplace=True)
```

```
[29]: df.drop(columns=['host_is_superhost', 'host_has_profile_pic',
↳ 'host_identity_verified', 'host_total_listings_count'], inplace=True)
```

```
[30]: df.drop(columns=['price'], inplace=True)
```

1.5.1 Data Preparation Summary:

Task: In the markdown cell below, document the data preparation steps you took. For each decision, explain why you made it. For example, why did you handle missing values the way you did? Why did you keep or remove certain features? If a preparation step depends on the model you selected (for example, scaling for KNN but not for a decision tree), explain that as well.

Missing values: To handle the missing values, I created dummy variables(`na`) indicating the missingness of the values before imputing the missing values with the column mean. This preserves the "missingness" while keeping the full dataset intact.

Redundant and irrelevant features: I dropped `'host_total_listings_count'` to reduce redundancy. I dropped `'host_is_superhost'`, `'host_has_profile_pic'`, and `'host_identity_verified'` because they had zero variance and little to no predictive power. I dropped the price column because keeping the exact variable used to create the label results in a trivial split and fake 100% accuracy. I also removed columns that had too many unique values (`'name'`, `'description'`, `'neighbourhood_overview'`, `'host_name'`, `'host_location'`, `'host_about'`, `'amenities'`) since they aren't usable as-is without additional processing (Natural Language Processing).

Categorical encoding: I applied one-hot encoding to `'neighbourhood_group_cleansed'` and `'room_type'` using `pd.get_dummies()` because scikit-learn's Decision Tree requires numerical input.

Model-specific preparation: Since I'm using a Decision Tree, I did not scale features or cap outlier columns. The model splits data using thresholds, making it robust to outliers and invariant to feature scaling.

1.6 Part 5: Train, Test, Evaluate, and Improve a Traditional Machine Learning Model

Now you will train, test and evaluate your model. You will also use model selection techniques to improve your model's performance by identifying the optimal hyperparameter configuration.

Task: In the code cells below, do the following:

1. Create labeled examples from the dataset
2. Create training and test sets out of the labeled examples
3. Train, test and evaluate your model
 - You must evaluate your model using accuracy and F1 score. Use `accuracy_score` and `f1_score` from `sklearn.metrics`. For the F1 score, use `average='binary'` since this is a binary classification problem. You will compare your model's performance to that of a neural network later in this capstone. Save the results of your evaluation metrics to variables for later comparison.
 - You may use additional evaluation metrics of your choosing.
4. Perform model selection through grid search cross-validation to identify optimal hyperparameter values for your model
5. Train, test and evaluate a final version of your model using the optimal hyperparameter configuration.
6. Interpret your model's outputs in the context of the business problem. Depending on the model you chose, this might mean:

- KNN: Describe what your model's performance metrics tell you about its behavior. For example: How does accuracy change as you vary k? What does that suggest about the structure of your data?
- Decision Tree: print or plot feature importances.
- Logistic Regression: print or plot the model coefficients.

Note: You can add code cells if needed by going to the Insert menu and clicking on Insert Cell Below in the drop-down menu.

```
[31]: bool_cols = df.select_dtypes(bool).columns
df[bool_cols] = df[bool_cols].astype(float)
```

```
[32]: # Create labeled examples from the dataset
y = df['price_category_num']
X = df.drop(columns = ['price_category', 'price_category_num'], axis=1)
```

```
[33]: # Create training and test sets out of the labeled examples
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=123)
print(X_train.shape)
print(X_test.shape)
```

```
(22417, 49)
```

```
(5605, 49)
```

```
[34]: # Train, test and evaluate your model
def train_test_DT(X_train, X_test, y_train, y_test, depth, leaf=1,
↳crit='entropy'):
    model = DecisionTreeClassifier(criterion = crit, max_depth = depth,
↳min_samples_leaf = leaf)
    model.fit(X_train, y_train)\

    class_label_predictions = model.predict(X_test)

    acc_score = accuracy_score(y_test, class_label_predictions)
    f1 = f1_score(y_test, class_label_predictions, average='binary')

    return acc_score, f1, model
```

```
[35]: # Perform model selection through grid search cross-validation (GridSearchCV)
# to identify optimal hyperparameter values for your model

# Create a range of hyperparameter values for 'max_depth'.
hyperparams_depth = [2**n for n in range(2,5)]

# Create a range of hyperparameter values for 'min_samples_leaf'.
hyperparams_leaf = [25*2**n for n in range(0,3)]
```



```

# Create parameter grid.
param_grid={'max_depth':hyperparams_depth, 'min_samples_leaf':hyperparams_leaf}
param_grid

model1 = DecisionTreeClassifier(class_weight='balanced')
grid = GridSearchCV(model1, param_grid, cv=5, scoring='f1')
grid_search = grid.fit(X_train, y_train)

# Inspect the best hyperparameters found
print("Optimal hyperparameters: {0}".format(grid_search.best_params_))
print("Best accuracy score: {0}".format(grid_search.best_score_))
print("Best F1 score (cross-validated): {0}".format(grid_search.best_score_))

```

Optimal hyperparameters: {'max_depth': 8, 'min_samples_leaf': 50}
Best accuracy score: 0.6594384424451949
Best F1 score (cross-validated): 0.6594384424451949

```

[36]: # Train, test and evaluate a final version of your model using the optimal_
      ↪hyperparameter values.
best_depth = grid_search.best_params_['max_depth']
best_leaf = grid_search.best_params_['min_samples_leaf']

final_acc_score, final_f1, final_model = train_test_DT(
    X_train, X_test, y_train, y_test, depth=best_depth, leaf=best_leaf
)

print("Final accuracy score: {0}".format(final_acc_score))
print("Final F1 score: {0}".format(final_f1))

```

Final accuracy score: 0.8299732381801963
Final F1 score: 0.6161900926298832

```

[37]: # Interpret your model's outputs
importances = pd.Series(final_model.feature_importances_, index=X_train.columns)
importances_sorted = importances.sort_values(ascending=False)

```

```

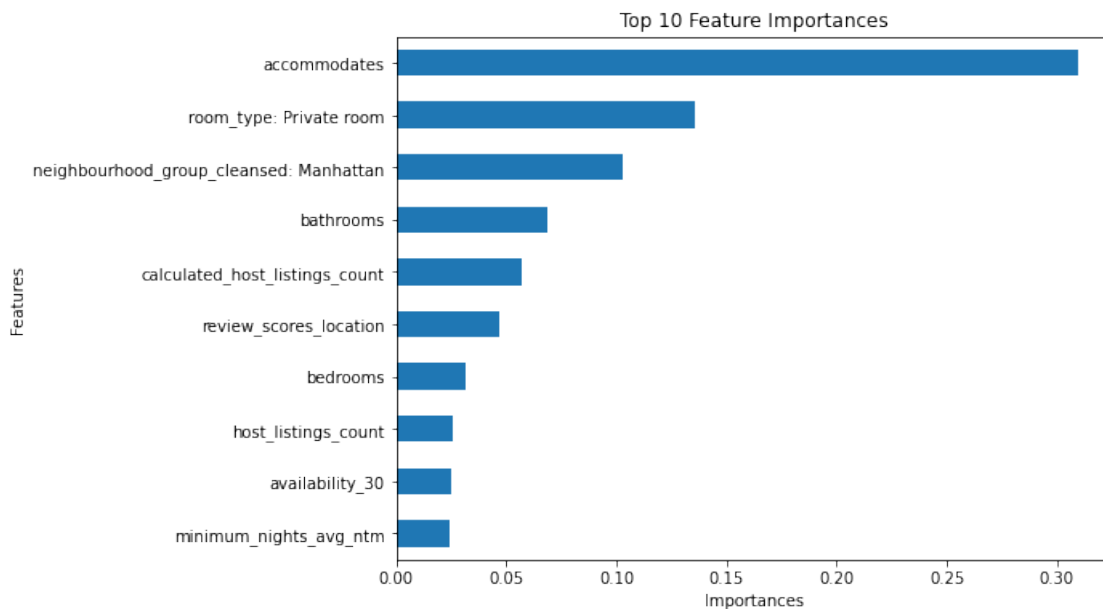
[38]: print(importances_sorted.head(10))

```

accommodates	0.309416
room_type: Private room	0.135429
neighbourhood_group_cleansed: Manhattan	0.103105
bathrooms	0.068596
calculated_host_listings_count	0.057191
review_scores_location	0.047188
bedrooms	0.031477
host_listings_count	0.025779
availability_30	0.025124
minimum_nights_avg_ntm	0.024018

dtype: float64

```
[39]: plt.figure(figsize=(8,6))
importances_sorted.head(10).plot(kind='barh')
plt.title('Top 10 Feature Importances')
plt.xlabel('Importances')
plt.ylabel('Features')
# most important feature is at the top
plt.gca().invert_yaxis()
plt.show()
```



1.6.1 Model Reflection:

Task: In the markdown cell below, answer the following questions:

1. Which model did you choose and why? Reference your problem and data characteristics in your explanation.
 2. What did you learn through the model selection process?
 3. How do you interpret your model's outputs? What do the predictions or coefficients or feature importances actually mean in the context of your problem?
 4. Are there any fairness or ethical concerns with your model? Who might be harmed by incorrect predictions, and are any groups more likely to be affected?
-
1. I chose a Decision Tree as my model because after performing exploratory data analysis, I noticed that there were extreme outliers that would be difficult to find and cap them all. The model splits data using thresholds, making it robust to outliers and invariant to feature scaling.

2. Through the model selection process, I set the grid search scoring parameter to 'f1' instead of accuracy given the class imbalance, so the model was tuned to actually identify high-priced listings rather than just favoring the majority class.
 3. The feature importances shows which features have the strongest effect on the label. From the model's output, 'accommodates', 'room_type: Private room', 'neighbourhood_group_cleansed: Manhattan' matter most when predicting if a listing is high-priced.
 4. Yes, the 'neighbourhood_group_cleansed: Manhattan' feature shows that location and socioeconomic status of a borough directly affects the prediction. Hosts in lower-income boroughs could be predicted as low-priced regardless of other features.
-

1.7 Unit 5 Checkpoint

Stop here. If you have completed Parts 1 through 5, you are done with the Unit 5 portion of this capstone.

Parts 6 and 7 require you to train and evaluate a neural network. You will learn about neural networks in the Unit 6 asynchronous content. Do not start Part 6 until you have completed that material and your lab facilitator has directed you to continue. Do not submit your work for grading until you complete Parts 6 and 7.

Make sure your notebook is saved before you close it.

1.8 Part 6: Train, Test, Evaluate and Improve a Neural Network

Before you write any code in Part 6, do this first.

Your notebook does not retain variables between sessions. All of your variables and everything else need to be restored to memory before any code below will work.

Go to **Kernel > Restart & Run All** to re-run Parts 1 through 5, then scroll back here to continue. If you skip this step, you will see a `NameError` on the first code cell below.

Now you will apply a neural network to the same problem and dataset. You will use Keras to build a feedforward neural network for binary classification.

The scaffolding below will walk you through the steps. Where you see a **Task**, fill in the code. Where you see a **Decision**, you are making an independent choice about your architecture or training process. For each decision, add a comment explaining what you chose and why.

1.8.1 Prepare Your Data for the Neural Network

Neural networks require all input features to be numeric and scaled. If your features are on very different scales (for example, one feature ranges from 0 to 90 and another from 0 to 99999), the model may train less effectively and have difficulty learning meaningful patterns from the data.

Before training your network, create scaled versions of your training and test data. Use `StandardScaler()` from `sklearn.preprocessing` to standardize your features:

Task: Complete the code cell below to fit the scaler on your training data, then transform both training and test sets. Save the results to new variables (for example, `X_train_scaled` and `X_test_scaled`) so your original data remains available for reference.

Note: Use your scaled data for all neural network steps below.

```
[40]: # Scale your data for the neural network

# Create the scaler
scaler = StandardScaler()

# Fit the scaler on the training data and transform the training data
X_train_scaled = scaler.fit_transform(X_train)

# Use the same scaler to transform the test data
X_test_scaled = scaler.transform(X_test)
```

1.8.2 Step 1: Define Your Model Architecture

You will use the Keras `Sequential` class to build your network. Your network should have:

- An input layer with the correct shape for your data
- At least two hidden layers using the `Dense` class
- An output layer appropriate for binary classification

Task: Create a `Sequential` model object and name it `nn_model`. Then construct and add each layer.

Decision: How many hidden layers will you use? How many units in each? What activation function will you use for the hidden layers? Add a comment explaining your choices.

```
[41]: # Get the number of features in your training data
n_features = X_train_scaled.shape[1]

# Create the neural network model
nn_model = keras.Sequential()

# Create the input layer and add the input layer to the 'nn_model' object
input_layer = keras.layers.InputLayer(input_shape=(n_features))
nn_model.add(input_layer)

# Create the hidden layers and add the hidden layers to the 'nn_model' object
# Decision: How many layers? How many units? What activation function?
# Add a comment explaining your choices.
```

```

# I will be using 2 hidden layers. Two hidden layers are sufficient to learn
↳ non-linear interactions between features without being
# overfitted.

# I will be using 64 and 32 units respectively for the hidden layers. Starting
↳ with 64 allows the network to learn different feature combinations.
# Reducing it to 32 for the second layer forces the network to learn the more
↳ important feature combinations. This prevents overfitting on the training
↳ data.

# I will be using the ReLU activation function for the hidden layers because
↳ the gradient stays strong for positive inputs
# instead of shrinking toward zero, which helps the network learn faster and
↳ train more reliably.

# For the output layer i will use the sigmoid activation function because the
↳ output values are either 0 or 1
# and the sigmoid function outputs a probability between 0 and 1

# First hidden layer
hidden_layer_1 = keras.layers.Dense(units=64, activation='relu')
nn_model.add(hidden_layer_1)

# Second hidden layer
hidden_layer_2 = keras.layers.Dense(units=32, activation='relu')
nn_model.add(hidden_layer_2)

# Create the output layer and add the output layer to the 'nn_model' object
# Use the correct number of units and activation function for binary
↳ classification
output_layer = keras.layers.Dense(units=1, activation='sigmoid')
nn_model.add(output_layer)

# Print a summary of your model
nn_model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	3200
dense_1 (Dense)	(None, 32)	2080
dense_2 (Dense)	(None, 1)	33
Total params: 5,313		

Trainable params: 5,313
Non-trainable params: 0

1.8.3 Step 2: Define the Optimization Function

Task: In the code cell below, create an optimizer object. Use stochastic gradient descent (SGD) with a learning rate of your choosing.

Decision: What learning rate will you use? Add a comment explaining your choice.

```
[42]: # Decision: What learning rate will you use? Add a comment explaining your
      ↪ decision.

      # I'm using 0.01 as my learning rate because it is large enough and trains in a
      ↪ reasonable number of epochs
      # but small enough to converge toward the minimum loss without overshooting.
      sgd_optimizer = keras.optimizers.SGD(learning_rate=0.01)
```

1.8.4 Step 3: Define the Loss Function

Task: In the code cell below, create a binary cross entropy loss function using `keras.losses.BinaryCrossentropy()`. Use the parameter `from_logits=False`.

```
[43]: loss_fn = keras.losses.BinaryCrossentropy(from_logits=False)
```

1.8.5 Step 4: Compile the Model

Task: In the code cell below, package the network architecture with the optimizer and the loss function using the `compile()` method. Use the accuracy evaluation metric.

```
[44]: nn_model.compile(optimizer=sgd_optimizer, loss=loss_fn, metrics=['accuracy'])
```

1.8.6 Step 5: Fit the Model to the Training Data

We will define our own callback class to output information from our model while it is training. Make sure you execute the code cell below so that it can be used in subsequent cells.

```
[45]: class ProgBarLoggerNEpochs(keras.callbacks.Callback):

      def __init__(self, num_epochs: int, every_n: int = 50):
          self.num_epochs = num_epochs
          self.every_n = every_n

      def on_epoch_end(self, epoch, logs=None):
```

```

if (epoch + 1) % self.every_n == 0:
    s = 'Epoch [{}/ {}]' .format(epoch + 1, self.num_epochs)
    logs_s = ['{}: {:.4f}' .format(k.capitalize(), v)
              for k, v in logs.items()]
    s_list = [s] + logs_s
    print(', '.join(s_list))

```

Task: Use the `fit()` method to fit your model to the training data. Save the result to variable `history`.

Use the `validation_split` parameter to reserve a portion of your training data for validation during training (a common choice is `validation_split=0.2`). After each epoch, the model is evaluated on this validation data, allowing you to monitor how well the model generalizes and helping you detect overfitting.

Also, use the the logger class defined above to track training progress.

Decision: How many epochs will you train for? Add a comment explaining your choice.

Note: This may take a while to run.

```

[46]: # Decision: How many epochs? Add a comment.
      # I will train using 50 epochs. Given the class imbalance I found during EDA
      ↪ more epochs helps the model
      # properly learn patterns in the minority class rather than stopping too early
      ↪ and defaulting to predicting
      # the majority class.

      t0 = time.time() # start time

      num_epochs = 50

      history = nn_model.fit(X_train_scaled, y_train, epochs=num_epochs,
      ↪ validation_split=0.2,
      verbose=0, callbacks=[ProgBarLoggerNEpochs(num_epochs,
      ↪ every_n=5)])

      t1 = time.time() # stop time

      print('Elapsed time: %.2fs' % (t1-t0))

```

Epoch [5/ 50], Loss: 0.3561, Accuracy: 0.8400, Val_loss: 0.3640, Val_accuracy: 0.8323

Epoch [10/ 50], Loss: 0.3392, Accuracy: 0.8474, Val_loss: 0.3545, Val_accuracy: 0.8363

Epoch [15/ 50], Loss: 0.3304, Accuracy: 0.8504, Val_loss: 0.3476, Val_accuracy: 0.8414

Epoch [20/ 50], Loss: 0.3233, Accuracy: 0.8550, Val_loss: 0.3443, Val_accuracy: 0.8430

```
Epoch [25/ 50], Loss: 0.3175, Accuracy: 0.8576, Val_loss: 0.3429, Val_accuracy: 0.8432
Epoch [30/ 50], Loss: 0.3125, Accuracy: 0.8590, Val_loss: 0.3406, Val_accuracy: 0.8414
Epoch [35/ 50], Loss: 0.3080, Accuracy: 0.8610, Val_loss: 0.3396, Val_accuracy: 0.8459
Epoch [40/ 50], Loss: 0.3039, Accuracy: 0.8648, Val_loss: 0.3392, Val_accuracy: 0.8466
Epoch [45/ 50], Loss: 0.2999, Accuracy: 0.8659, Val_loss: 0.3402, Val_accuracy: 0.8412
Epoch [50/ 50], Loss: 0.2966, Accuracy: 0.8683, Val_loss: 0.3389, Val_accuracy: 0.8477
Elapsed time: 46.72s
```

1.8.7 Step 6: Visualize Training Performance

Task:

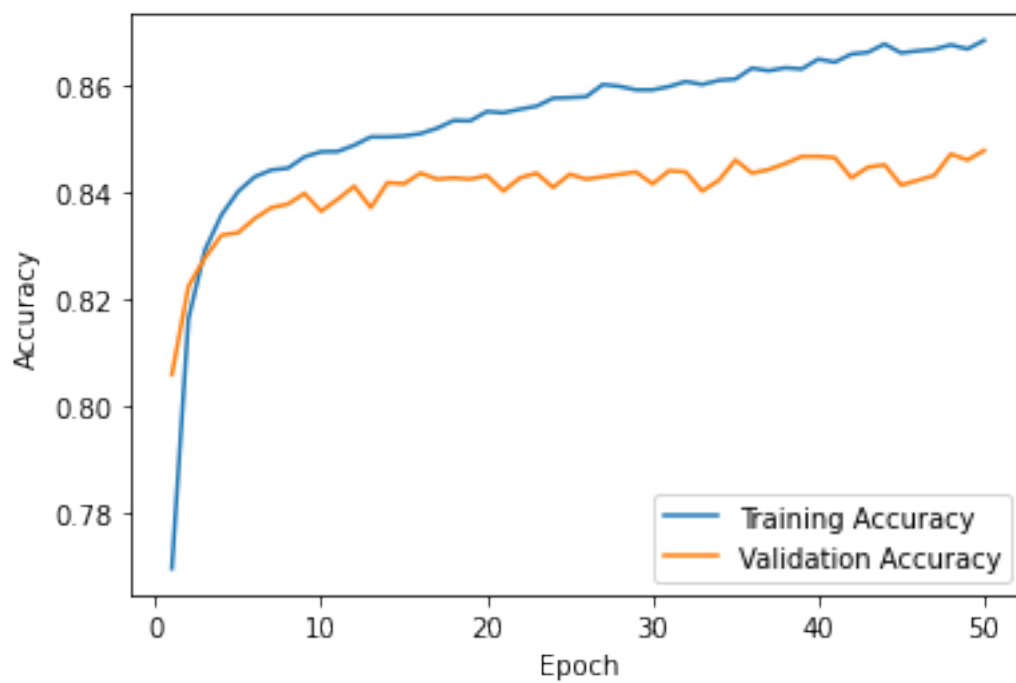
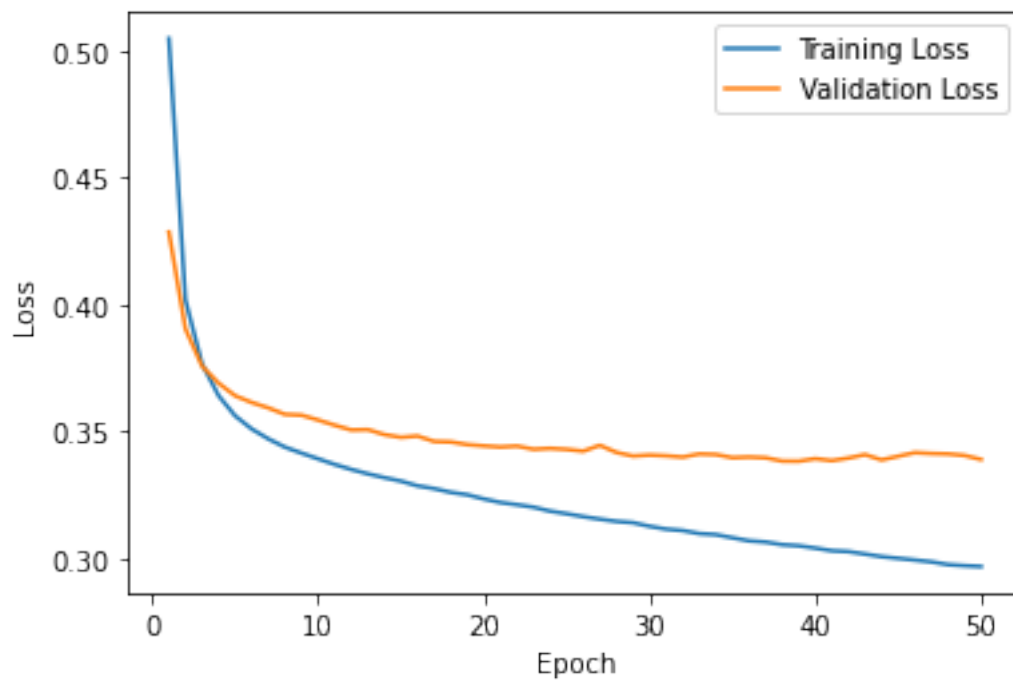
Create two plots to visualize the model's performance over time: 1. Training loss and validation loss over epochs, on the same axes. 2. Training accuracy and validation accuracy over epochs, on the same axes.

Label your axes and include a legend.

Use the `history` object returned by `fit()` to obtain this information.

```
[47]: # Plot training loss and validation loss over epochs
plt.plot(range(1, num_epochs + 1), history.history['loss'], label='Training Loss')
plt.plot(range(1, num_epochs + 1), history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Plot training accuracy and validation accuracy over epochs
plt.plot(range(1, num_epochs + 1), history.history['accuracy'], label='Training Accuracy')
plt.plot(range(1, num_epochs + 1), history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

1.8.8 Step 7: Evaluate the Model's Performance on the Test Set

Task: Use your neural network to generate predictions on the test set and evaluate its performance using accuracy and F1 score. Use `nn_model.predict()` to generate predictions. Since `nn_model.predict()` returns probabilities, apply a threshold of 0.5 to convert probabilities into binary class predictions before computing your metrics. Save your accuracy and F1 score results to clearly named variables so you can compare them to the results from your previous model. Print the results.

```
[48]: # Generate predictions from your neural network using your scaled test data
# nn_model.predict() returns probabilities - apply a threshold of 0.5 to get
      ↪ class labels
probability_predictions = nn_model.predict(X_test_scaled)
class_label_predictions=[]
for i in range(0,len(y_test)):
    if probability_predictions[i] >= 0.5:
        class_label_predictions.append(1)
    else:
        class_label_predictions.append(0)
```

```
[49]: # Compute accuracy and F1 score for the neural network and print the results

nn_accuracy = accuracy_score(y_test, class_label_predictions)
nn_f1 = f1_score(y_test, class_label_predictions, average='binary')

print("Neural Network Accuracy: {0}".format(nn_accuracy))
print("Neural Network F1 Score: {0}".format(nn_f1))
```

```
Neural Network Accuracy: 0.8499553969669937
Neural Network F1 Score: 0.667194301543332
```

Neural Network Reflection: Task: In the markdown cell below, answer the following questions:

1. Walk through the architecture decisions you made: number of layers, number of units, activation functions, learning rate, and number of epochs. Why did you make each of those choices?
 2. What did your training curves tell you? Did you see any signs of overfitting or underfitting?
 3. How did your neural network perform on the test set? Report your accuracy and F1 score here and note whether the result surprised you given what your training curves showed.
-
1. As detailed in my code comments, I chose 2 hidden layers (64 and 32 units) with ReLU to learn non-linear interactions without overfitting, a learning rate of 0.01 to converge smoothly without overshooting, and 50 epochs to ensure the model learns the minority class patterns. The output layer uses a sigmoid activation to output probabilities for this binary classification task.
 2. The training curves show signs of mild overfitting, as the training loss continues to decrease while the validation loss stabilizes after about 20 epochs. However, the gap between the training and validation accuracy remains very small, indicating the model still generalizes

reasonably well to unseen data.

3. The neural network achieved an accuracy of 0.85 and an F1 score of 0.67 on the test set. This result was not surprising, as the test accuracy perfectly matches the validation accuracy seen in the training curves, confirming the model generalized well.

1.9 Part 7: Compare Your Models and Reflect

You have now applied two different approaches to the same problem. In this final section, you will put those results side by side and reflect on what you learned.

1.9.1 Results Summary

Task: In the code cell below, create a summary table using a Pandas DataFrame that displays each evaluation metric for both models side by side. Use the variables you created for the accuracy and F1 score metrics. The table should make it easy to compare performance at a glance across every metric you computed.

```
[50]: # Build a side-by-side comparison of your two models using the metric variables
# you created. For example:
#
# results = pd.DataFrame({
#     'Metric': ['Accuracy', 'F1 Score'],
#     'KNN Model': [knn_accuracy, knn_f1],
#     'Neural Network': [nn_accuracy, nn_f1]
# })
# print(results.to_string(index=False))
#
results = pd.DataFrame({
    'Metric': ['Accuracy', 'F1 Score'],
    'Decision Tree': [final_acc_score, final_f1],
    'Neural Network': [nn_accuracy, nn_f1]
})
print(results.to_string(index=False))
```

Metric	Decision Tree	Neural Network
Accuracy	0.829973	0.849955
F1 Score	0.616190	0.667194

1.9.2 Comparative Analysis

Task: In the markdown cell below, write a comparative analysis that addresses the following.

1. **Performance comparison.** How did the two models perform relative to each other? Which metrics improved, which stayed the same, and which got worse?

2. **Was the added complexity worth it?** Neural networks are more complex to build, tune, and interpret. Given the performance difference you observed, do you think the neural network justified that added complexity for this problem?
 3. **Recommendation.** If you were presenting this work to your company and their client as described in the business brief, which model would you recommend deploying and why? Consider not just performance but also interpretability, training time, and what the costs of different types of errors look like for that specific client.
 4. **What you would do next.** If you had more time, what would you try to improve your results? This could include trying different architectures, additional preprocessing steps, different features, or techniques to address class imbalance. Be specific.
1. The neural network slightly outperformed the decision tree across the board. The Decision Tree achieved an accuracy of 0.83 and an F1 score of 0.62, while the Neural Network improved on both metrics, reaching an accuracy of 0.85 and an F1 score of 0.67.
 2. Given the slight difference in metrics, I would say the added complexity wasn't worth it. The Decision Tree model had highly comparable results with significantly less tuning and computational cost.
 3. If I was presenting this work to my company, I would recommend deploying the Decision Tree because it is much easier to explain to non-technical users. The model is much faster to train and requires significantly less computing power than a neural network. Since the performance difference is so small, the simpler model is the most practical choice for this business problem.
 4. If I had more time, I would try testing a wider range of hyperparameter values to see if I could improve the F1 score. For the Decision Tree, I would test more combinations of max depth and minimum leaf size. For the neural network, I would experiment with different learning rates, the number of epochs, and the number of units in the hidden layers to find the absolute best configuration.
-

1.10 AI Use Attestation

Reflect honestly on how you used AI tools during this capstone. You are expected to have used AI somewhere in your workflow, and your reflection on that use is what will be evaluated: How clearly you describe your choices, how you verified your work, and what you learned from the process. If you chose not to use AI, explain why. Answer each question in the markdown cell below.

1. Where and at what stages of this capstone did you use AI tools, for example, Claude during brainstorming, coding, or debugging? If you chose not to use AI, explain why.
2. Identify one part of the capstone that required the most effort or thought. What made it challenging, and how did you work through it, with AI or without AI? If you used AI at this point, feel free to share a prompt that worked well or one that did not land the way you expected.
3. How did you verify that your work was correct? What did you look for to catch a mistake, whether it came from AI output or your own reasoning?
4. What is one thing you would do differently next time, either in how you approached the capstone or in how you used AI during it?

1. Yes. I used AI as a debugging tool during Part 6 when my neural network failed to train due to a `ValueError` related to data types. I was unfamiliar with TensorFlow's strict requirements for tensor conversion and didn't know how to resolve the error on my own. I pasted the full error message into an AI tool, which explained that boolean columns cannot be directly converted to tensors and suggested casting them to floats first. Based on that explanation, I implemented the following fix myself: `bool_cols = df.select_dtypes(bool).columns` `df[bool_cols] = df[bool_cols].astype(float)` I then tested the code to confirm it resolved the error. The AI helped me understand why the error occurred and pointed me toward a solution, but I wrote, executed, and validated the final code independently.
2. The most challenging part was preparing the data for the neural network. Unlike the Decision Tree, I had to scale all features and convert boolean columns to floats because TensorFlow cannot process mixed data types. This was frustrating when my model failed to train due to a tensor conversion error. I worked through it by carefully checking each column's dtype and applying `StandardScaler` only after ensuring all inputs were numeric. While AI helped me understand why the error occurred, I implemented and tested the fix myself, which reinforced the importance of thorough data preprocessing before modeling.
3. To verify my work was correct I would investigate the output and see if it was printing out something related to the expected format. I also used some of the previous labs and coding assignments to make sure I was going in the right track.
4. One thing I would have done differently is to ask for help during office hours because I think that would have been helpful to clarify some of the decisions I made throughout the completion of the capstone project